

Documento Proyecto DPOO - Entrega 3

Integrantes:

Carolina Ceballos Gómez - 202510267

Andrés Santiago Rodríguez Salazar - 202513997

Sebastián Felipe Angarita Rico - 202516784

A. Diagrama Diseño UML general:

Al menos un diagrama de clases que incluya todas las clases del sistema, incluyendo sus relaciones, atributos y métodos. Los diagramas deben cubrir tanto la interfaz como la parte del diseño dedicado a la lógica de dominio.

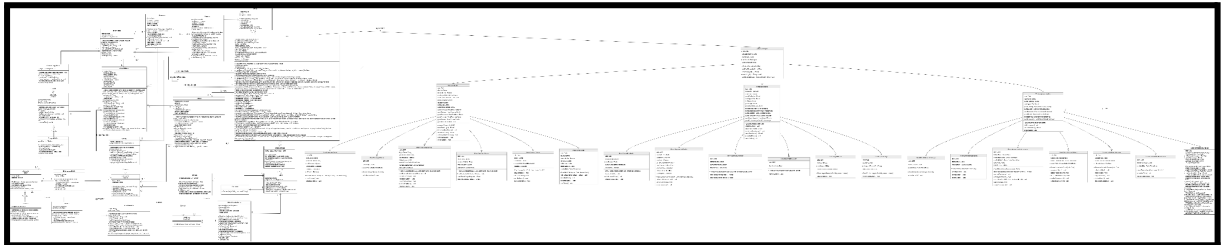


Imagen y archivo fuente de draw.io se encuentran en Entrega 3 - Docs

B. Diagrama Diseño UML detallado (nivel final)

Un diagrama de clases de alto nivel, que incluya todas las clases del sistema y sus relaciones, diferente al A ya que no se menciona interfaz: Objetos/Elementos, atributos, métodos, relaciones y cardinalidades del modelo.

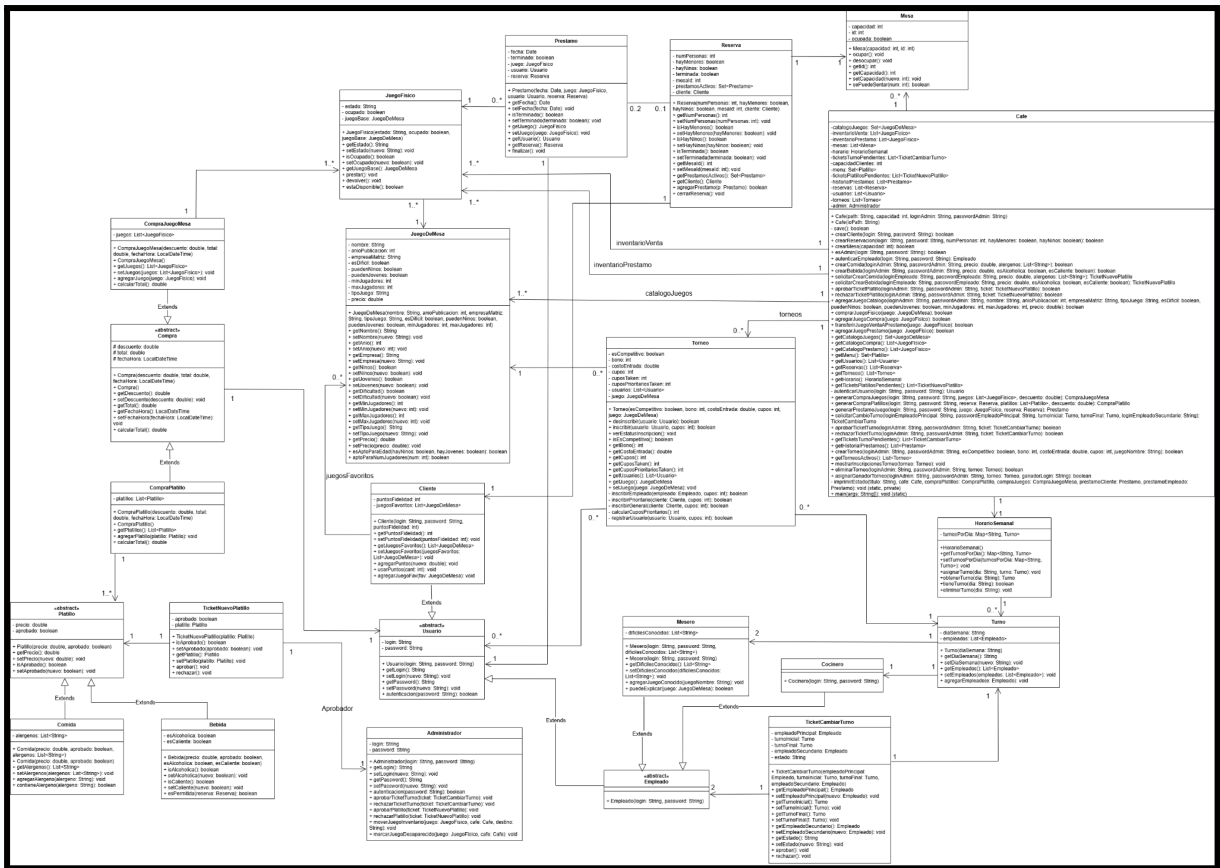


Imagen y archivo fuente de draw.io se encuentran en Entrega 3 - Docs

Este diagrama evidencia la mejora y evolución del modelo que se presentó en la entrega 1. Para esta versión, se realizaron ajustes importantes basados en la retroalimentación recibida, especialmente en lo relacionado con la implementación de persistencia y la mejora en las cardinalidades. Se usó la interfaz Serializable en las clases para garantizar la persistencia del estado del sistema en los archivos, y Administrador pasó a ser una clase independiente. Gracias a las relaciones, cardinalidades, el hecho de que Cafe actúa correctamente como núcleo del sistema, hacen que el diagrama refleje los requerimientos funcionales del sistema.

C. UN DIAGRAMA DE CLASES DE ALTO NIVEL DE LA INTERFAZ:

Un diagrama de clases de alto nivel de la interfaz, que muestre qué elementos se incluye y cómo se relacionan con los elementos del dominio:

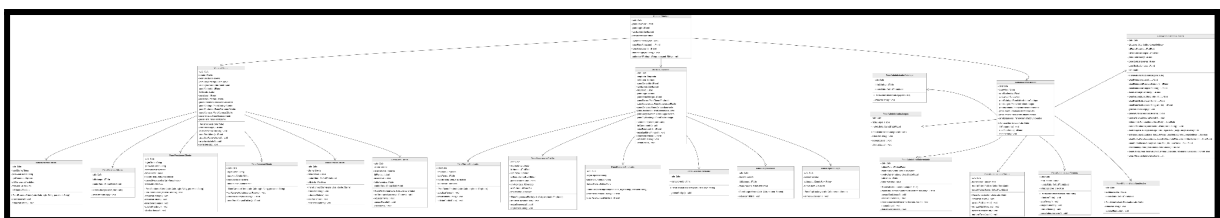


Imagen y archivo fuente de draw.io se encuentran en Entrega 3 - Docs

1. Introducción

Este documento describe el diseño y la implementación de la interfaz gráfica de usuario (GUI) para el sistema de gestión del Board Game Cafe "Dulces & Dados". La aplicación permite a clientes, empleados y administradores interactuar con el sistema a través de una interfaz intuitiva basada en Swing, reemplazando la anterior interfaz de consola. Se han implementado todas las funcionalidades previamente soportadas y se han añadido nuevas visualizaciones gráficas (diagrama de pastel, barras y líneas) para facilitar la comprensión de datos clave por parte del administrador.

2. Diseño General de la Interfaz

La interfaz gráfica se organiza en torno a una ventana principal (VentanaPrincipal) que permite la selección del rol (Cliente, Empleado, Administrador). Una vez autenticado, se abre la ventana correspondiente a cada rol, la cual contiene un menú lateral con las opciones disponibles y un área central donde se muestra el contenido dinámico.

- VentanaPrincipal

Análisis

Ventana de entrada de la aplicación. Presenta un panel con tres botones para seleccionar el rol (Cliente, Empleado, Administrador). Utiliza CardLayout para alternar entre el panel de selección y el panel de login (para empleado y admin). Para cliente, abre directamente VentanaCliente. Al seleccionar empleado o admin, muestra el panel de login y autentica antes de abrir la ventana correspondiente.

Atributos

- cafe: Cafe
- panelSeleccion: JPanel
- panelLogin: JPanel
- cardLayout: CardLayout
- rolSeleccionado: String

Métodos

- + VentanaPrincipal(cafe: Cafe)

- crearPanelSeleccion(): JPanel
- crearPanelLogin(): JPanel
- mostrarLogin(rol: String): void
- autenticarYRol(login: String, password: String): void

- **VentanaAdministrador**

Análisis

Ventana principal para el administrador autenticado. Presenta un menú lateral con opciones para gestionar juegos, inventario, tickets, torneos y empleados. Utiliza CardLayout en el área central para cambiar entre los diferentes paneles. Al cerrar la ventana, guarda automáticamente los cambios y cierra la sesión.

Atributos

- cafe: Cafe
- loginAdmin: String
- panelContenido: JPanel
- cardLayout: CardLayout
- panelCatalogo: PanelAdministradorCatalogo
- panelJuegos: PanelAdministradorJuegos
- panelInventario: PanelAdministradorInventario
- panelTickets: PanelAdministradorTickets
- panelTorneos: JPanel
- panelEmpleados: JPanel
- panelGraficas: JPanel

Métodos:

- + VentanaAdministrador(cafe: Cafe)
- initComponents(): void
- crearPanelMenu(): JPanel
- cerrarSesion(): void

Paneles utilizados (metodos y atributos en el UML general):

PanelAdministradorCatalogo: Este panel muestra el catálogo completo de juegos de mesa en una tabla con columnas: Nombre, Tipo, Año, Empresa, Precio, Min/Max, Difícil, Niños, Jóvenes. Incluye un botón para refrescar la tabla. Los datos se obtienen directamente de `cafe.getCatalogoJuegos()` y se muestran en un `JTable` no editable.

PanelAdministradorEmpleados: Permite al administrador ver la lista de empleados registrados (meseros y cocineros) y registrar nuevos empleados. Al registrar, solicita login, contraseña y rol, validando que el login no exista previamente.

PanelAdministradorInventario: Gestiona los inventarios de venta y préstamo usando un JTabbedPane. Permite mover juegos entre catálogos, reparar juegos dañados, marcar juegos como desaparecidos y agregar nuevas copias físicas de juegos existentes.

PanelAdministradorJuegos: Permite al administrador gestionar el catálogo de juegos: ver, agregar y eliminar juegos. Al agregar, presenta un formulario con todos los atributos necesarios (nombre, año, empresa, tipo, dificultad, restricciones de edad, jugadores, precio).

PanelAdministradorTickets: Maneja dos tipos de tickets pendientes de aprobación: sugerencias de platillos y cambios de turno. Usa un JTabbedPane para separar ambas secciones. Permite aprobar o rechazar cada ticket, actualizando el estado correspondiente.

PanelAdministradorTorneos: Gestiona los torneos del café. Permite crear nuevos torneos (con juego asociado, bono, costo, cupos), asignar ganadores, eliminar torneos y consultar inscripciones detalladas. Los torneos pueden ser competitivos o amistosos.

PanelAdministradorGraficas: Este panel implementa las tres visualizaciones gráficas solicitadas en el proyecto 3, utilizando la librería JFreeChart. Permite al administrador generar:

- Diagrama de pastel: Muestra la distribución de copias de un juego entre inventario de préstamo y venta.
- Diagrama de líneas: Muestra la evolución del número de reservas durante una semana específica.
- Diagrama de barras: Muestra las ventas (sin impuestos) de cafetería vs juegos en un periodo de 5 días.

El panel está organizado en tres secciones con formularios para seleccionar el juego, la fecha de inicio de semana y la fecha de inicio del período de ventas. Cada sección tiene su propio espacio para mostrar la gráfica generada.

- VentanaCliente

Análisis

Ventana principal para clientes. Utiliza dos CardLayout: uno para alternar entre login/registro y menú principal, y otro para alternar entre los paneles de contenido (reservas, catálogo, préstamos, compras, torneos, perfil). Al iniciar, muestra la pantalla de login/registro. Después

del login exitoso, muestra el menú principal con el estado del cliente (puntos y reserva activa).

Atributos

- cafe: Cafe
- cliente: Cliente
- reservaActiva: Reserva
- cardLayoutPrincipal: CardLayout
- cardLayoutContenido: CardLayout
- panelContenido: JPanel
- lblEstado: JLabel
- panelLogin: JPanel
- panelMenuPrincipal: JPanel
- panelReservas: PanelReservasCliente
- panelCatalogo: PanelCatalogoCliente
- panelPrestamos: PanelPrestamosCliente
- panelCompras: PanelComprasCliente
- panelTorneos: PanelTorneosCliente
- panelPerfil: PanelPerfilCliente

Métodos:

- + VentanaCliente(cafe: Cafe)
- crearPanelLogin(): JPanel
- inicializarMenuPrincipal(): void
- crearPanelMenu(): JPanel
- actualizarReservaActiva(): void
- actualizarEstado(): void
- cerrarSesion(): void

Paneles utilizados (metodos y atributos en el UML general):

PanelPerfilCliente: Muestra al cliente su información personal: login, puntos de fidelidad, estado de reserva activa y lista de juegos favoritos. Permite agregar nuevos juegos a favoritos (desde un combo box con todos los juegos del catálogo) y usar puntos de fidelidad como descuento (con validación de cantidad disponible). Incluye botón para refrescar la información.

PanelPrestamosCliente: Permite al cliente seleccionar hasta 2 juegos disponibles del catálogo de préstamo, solicitar el préstamo y devolver los juegos prestados. Incluye validaciones de reserva activa, límite de 2 préstamos por reserva, y disponibilidad de juegos. Muestra estado actual de préstamos activos. Tiene logs de depuración para identificar fallos.

PanelReservasCliente: Permite al cliente crear una reserva indicando número de personas, presencia de menores y niños. Verifica disponibilidad de mesas y capacidad del café. Muestra el estado de la reserva activa actual (mesa, personas, menores, niños, préstamos activos). Incluye validación para no crear múltiples reservas activas.

PanelTorneosCliente: Muestra al cliente los torneos disponibles con información de juego, tipo, bono, costo y cupos disponibles. Permite inscribirse (hasta 3 participantes por torneo) y desinscribirse. Valida cupos disponibles y prioridad de fanáticos (20% de cupos). Muestra las inscripciones actuales del cliente.

PanelCatalogoCliente: Este panel muestra al cliente los juegos disponibles para préstamo en una tabla. Muestra el nombre, tipo, rango de jugadores, precio y disponibilidad. Los datos se obtienen de `cafe.getCatalogoPrestamo()`. Incluye un botón para refrescar la tabla.

PanelComprasCliente: Permite al cliente comprar juegos y platillos usando un `JTabbedPane`. Para comprar juegos, muestra una tabla con checkboxes para seleccionar múltiples juegos del catálogo de venta. Para comprar platillos, requiere una reserva activa y valida restricciones de edad y tipo de juego. Al comprar, se acumulan puntos de fidelidad.

- VentanaEmpleado

Análisis

Ventana principal para empleados (meseros y cocineros). Similar a `VentanaCliente`, presenta un menú lateral con opciones según el rol (consultar turnos, sugerir platillo, comprar con descuento, generar código de descuento, y para meseros: ver juegos que conoce, explicar juego). Utiliza `CardLayout` para cambiar entre paneles.

Atributos

- `cafe`: `Cafe`
- `empleado`: `Empleado`
- `esMesero`: `boolean`
- `panelContenido`: `JPanel`
- `cardLayout`: `CardLayout`
- `lblEstado`: `JLabel`
- `panelLogin`: `JPanel`
- `panelMenuPrincipal`: `JPanel`
- `panelTurnos`: `PanelTurnosEmpleado`
- `panelSugerencias`: `PanelSugerenciasPlatillo`
- `panelCompras`: `PanelComprasEmpleado`
- `panelDescuento`: `PanelGenerarDescuento`
- `panelJuegosMesero`: `PanelJuegosMesero` (solo si es mesero)

- panelExplicarJuego: PanelExplicarJuego (solo si es mesero)

Métodos

- + VentanaEmpleado(cafe: Cafe)
- initComponents(): void
- crearPanelLogin(): JPanel
- inicializarMenuPrincipal(): void
- crearPanelMenu(): JPanel
- actualizarEstado(): void
- cerrarSesion(): void

Paneles utilizados (metodos y atributos en el UML general):

PanelComprasEmpleado: Permite al empleado comprar juegos y platillos con 20% de descuento. Para comprar juegos, selecciona múltiples juegos del catálogo de venta. Para comprar platillos, debe seleccionar un cliente con reserva activa. El descuento se aplica automáticamente al total.

PanelExplicarJuego: Permite a un mesero verificar si puede explicar un juego difícil a un cliente. Muestra un combo box con los juegos difíciles del catálogo y, al seleccionar uno, verifica si el mesero lo tiene en su lista de juegos conocidos (dificilesConocidos). Muestra el resultado en un área de texto.

PanelGenerarDescuento: Permite a un empleado generar un código de descuento del 10% para compartir con clientes. El código se genera con el formato DESC10_{login}_{timestamp}. Muestra un área de información y un botón que abre un diálogo con el código generado.

PanelJuegosMesero: Muestra al mesero los juegos difíciles del catálogo y si puede explicarlos o no. La tabla tiene dos columnas: "Juego" y "¿Puede Explicar?". Verifica si el nombre del juego está en la lista difícilesConocidos del mesero. Incluye un botón para refrescar y un área de información.

PanelSugerenciasPlatillo: Permite a los empleados sugerir nuevos platillos al administrador. El formulario cambia según el tipo (comida o bebida). Para bebidas: opciones alcohólica/caliente. Para comidas: lista de alérgenos. Valida que los campos no estén vacíos y envía el ticket al administrador para aprobación.

PanelTurnosEmpleado: Muestra al empleado el horario semanal con todos los turnos y marca con "✓" sus turnos asignados. Permite solicitar cambios de turno (generales o intercambio con otro empleado). Valida que los turnos existan y que el empleado esté asignado al turno actual. La solicitud queda pendiente de aprobación del administrador

3. Estructura de Paquetes

La aplicación mantiene la separación entre la lógica de negocio y la interfaz gráfica:

- control: Contiene la clase Café (núcleo del sistema), Administrador y las consolas (ahora obsoletas para la GUI, pero se mantienen por compatibilidad).
- usuarios: Jerarquía de usuarios (Usuario, Cliente, Empleado, Mesero, Cocinero).
juego: Clases relacionadas con juegos (JuegoDeMesa, JuegoFisico, Prestamo, Torneo).
- reservacion: Clases Mesa y Reserva.
- cafeteria: Clases Platillo, Comida, Bebida, TicketNuevoPlatillo.
- compras: Clases Compra, CompraJuegoMesa, CompraPlatillo.
- horario: Clases HorarioSemanal, Turno, TicketCambiarTurno.
- ui: Contiene todas las clases de la interfaz gráfica.
- ui.panels: Subpaquete con los paneles reutilizables para cada funcionalidad.

4. Descripción de las Ventanas Principales

4.1 VentanaPrincipal

Propósito: Ventana de entrada que permite seleccionar el rol y autenticarse.

Componentes:

- CardLayout para alternar entre panel de selección y panel de login.
- Panel de selección con 3 botones: "Cliente", "Empleado", "Administrador".
- Panel de login con campos de texto para login y contraseña.

Flujo:

1. El usuario selecciona un rol.
2. Se muestra el panel de login.

3. Al autenticar, se abre la ventana correspondiente y se cierra VentanaPrincipal.

4.2 VentanaCliente

Propósito: Ventana principal para los clientes autenticados.

Estructura:

- CardLayout principal que alterna entre el panel de login/registro y el menú principal.
- Menú lateral con opciones: "Crear Reserva", "Ver Catálogo", "Préstamos", "Compras", "Torneos", "Mi Perfil".
- Área central (CardLayout) donde se muestran los paneles correspondientes.
- Panel inferior con estado del cliente (puntos y reserva activa).

Paneles asociados:

- PanelReservasCliente: Crear y finalizar reservas.
- PanelCatalogoCliente: Ver juegos disponibles.
- PanelPrestamosCliente: Seleccionar, solicitar y devolver juegos en préstamo.
- PanelComprasCliente: Comprar juegos y platillos.
- PanelTorneosCliente: Consultar, inscribirse y desinscribirse de torneos.
- PanelPerfilCliente: Ver puntos, favoritos y usar puntos de fidelidad.

4.3 VentanaEmpleado

Propósito: Ventana para empleados (meseros y cocineros).

Estructura:

- Similar a VentanaCliente, con menú lateral adaptado al rol.
- Opciones comunes: "Consultar Turnos", "Sugerir Platillo", "Comprar (20% descuento)", "Generar Código Descuento".

- Opciones adicionales para meseros: "Juegos que Conozco", "Explicar Juego Difícil".

Paneles asociados:

- PanelTurnosEmpleado: Consultar turnos y solicitar cambios.
- PanelSugerenciasPlatillo: Sugerir nuevos platillos.
- PanelComprasEmpleado: Comprar juegos con descuento.
- PanelJuegosMesero: Ver juegos difíciles que conoce.
- PanelExplicarJuego: Verificar si puede explicar un juego.

4.4 VentanaAdministrador

Propósito: Ventana para el administrador.

Estructura:

- Menú lateral con opciones: "Gestionar Juegos", "Gestionar Inventario", "Resolver Tickets", "Gestionar Torneos", "Gestionar Empleados", "Estadísticas".

Paneles asociados:

- PanelAdministradorJuegos: CRUD de juegos en el catálogo.
- PanelAdministradorInventario: Gestionar inventarios de venta y préstamo (mover, reparar, marcar desaparecido, agregar copias).
- PanelAdministradorTickets: Aprobar/rechazar sugerencias de platillos y cambios de turno.
- PanelAdministradorTorneos: Crear torneos, asignar ganadores, eliminar torneos, consultar inscripciones.
- PanelAdministradorEmpleados: Registrar nuevos empleados.
- PanelGraficaPastel, PanelGraficaBarras, PanelGraficaLineas: Visualizaciones gráficas.

5. Visualizaciones Gráficas

El administrador ha solicitado tres visualizaciones gráficas para facilitar la comprensión de datos. Estas se implementan en un único panel `PanelAdministradorGraficas` utilizando la librería `JFreeChart`.

5.1 Diagrama de Pastel: Distribución de Copias (Venta vs Préstamo)

Propósito: Mostrar la proporción entre copias disponibles para la venta y copias disponibles para préstamo de un juego seleccionado.

Implementación: Método `crearGraficaTortaJuegos()` que genera un `ChartPanel` con un `JFreeChart` de tipo `pie`.

Datos: Se obtienen de `cafe.getCatalogoCompra()` y `cafe.getCatalogoPrestamo()`, filtrando por el juego seleccionado en un `JComboBox`.

Flujo: El administrador selecciona un juego del desplegable y hace clic en "Generar gráfica". El método `generarGraficaJuego()` cuenta las copias en ambos inventarios y genera el gráfico.

5.2 Diagrama de Barras: Ventas Discriminadas (Período de 5 Días)

Propósito: Mostrar las ventas de cafetería vs ventas de juegos en un período de 5 días, descontando impuestos.

Implementación: Método `crearGraficaBarrasVentas()` que genera un `ChartPanel` con un `JFreeChart` de tipo `barra`.

Datos: Se obtienen de `cafe.getCompras()`, filtrando por fecha y tipo (`CompraPlatillo` vs `CompraJuegoMesa`). Se descuentan impuestos usando `descontarImpuestos()` (IVA del 19%).

Eje X: Días del período. Eje Y: Valor de ventas sin impuestos.

Flujo: El administrador ingresa una fecha inicial y se generan automáticamente los 5 días siguientes. El método `generarGraficaComprasPeriodo()` procesa las compras y genera el gráfico.

5.3 Diagrama de Líneas: Evolución de Reservas (Semana)

Propósito: Mostrar la evolución del número de reservas durante una semana específica.

Implementación: Método `crearGraficaLineasReservas()` que genera un `ChartPanel` con un `JFreeChart` de tipo `línea`.

Datos: Se obtienen de `cafe.getReservas()`, agrupando por día de la semana seleccionada.

Eje X: Días de la semana (lunes a domingo). Eje Y: Número de reservas.

Flujo: El administrador ingresa la fecha de un lunes. El método `generarGraficaReservasSemana()` calcula la semana completa y, usando `contarReservasDia()`, genera el gráfico de evolución.

6. Justificaciones de Diseño

6.1 Uso de CardLayout para la Navegación

Se utilizó `CardLayout` tanto en las ventanas principales como en los paneles de contenido para permitir una navegación fluida entre las diferentes vistas. Esto evita la creación y destrucción constante de ventanas, mejorando el rendimiento y la experiencia del usuario.

6.2 Separación en Paneles

Cada funcionalidad principal se encapsula en su propio `JPanel` (ej: `PanelReservasCliente`, `PanelAdministradorJuegos`). Esto sigue el principio de responsabilidad única (SRP) y facilita el mantenimiento, prueba y reutilización del código.

6.3 Comunicación entre Paneles y el Modelo

Cada panel recibe una referencia al objeto `Cafe` (el núcleo del sistema) en su constructor. Esto permite acceder directamente a los métodos de negocio sin necesidad de intermediarios complejos. Aunque esto crea un acoplamiento, simplifica la implementación y es adecuado para el alcance del proyecto.

6.4 Uso de CardLayout para Login y Menú Principal en VentanaCliente

Para simplificar el flujo, `VentanaCliente` maneja internamente el login/registro antes de mostrar el menú principal. Esto evita tener que abrir ventanas adicionales y centraliza la lógica de autenticación del cliente.

6.5 Persistencia

Se mantuvo la estrategia de persistencia del Proyecto 2: serialización de Java. Cada ventana guarda el estado del `Cafe` al cerrarse, y la aplicación principal guarda al cerrar. Esto asegura que los datos no se pierdan entre ejecuciones.

7. Instrucciones de Ejecución

1. Importar el proyecto en Eclipse.
2. Ejecutar la clase control.AplicacionGUI.
3. La aplicación cargará (o creará) el archivo escenario1.txt para persistencia.
4. Si es la primera ejecución, se recomienda crear datos de prueba desde la interfaz de administrador.
5. Para autenticación inicial:
6. Administrador: login = "admin", password = "admin" (o los configurados al crear el archivo).
7. Cliente/Empleado: Registrarse desde la opción correspondiente.

8. Diagrama de secuencia:

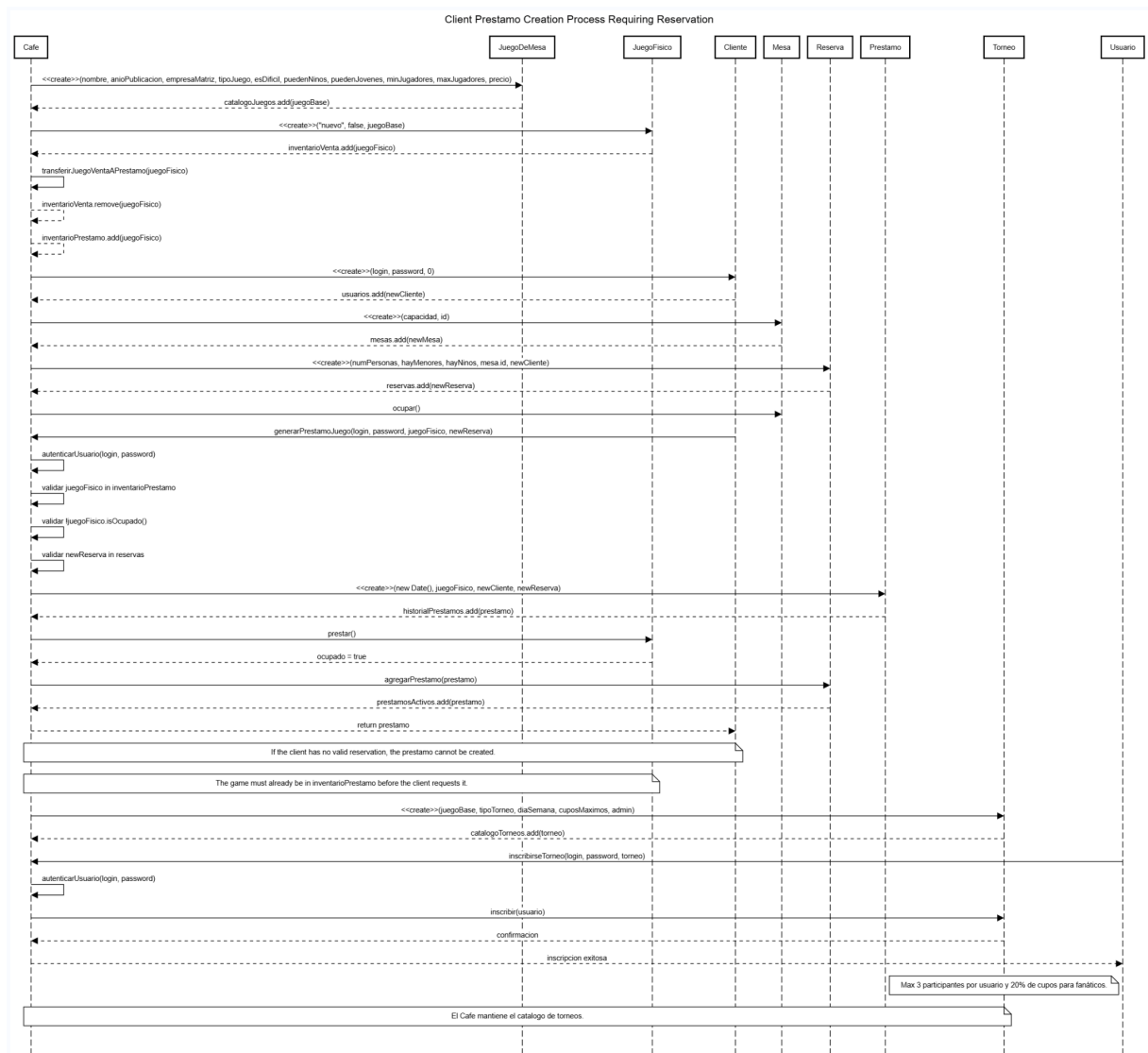


Imagen y archivo fuente de draw.io se encuentran en Entrega 3 - Docs